

Optimisasi Perolehan Produk *E-Commerce*: Pencarian Semantik Berbasis *Dense-Retrieval*, Pemeringkatan Ulang dengan *Cross-Encoder* dan Pembelajaran Mesin, serta Integrasi Pencarian Visual untuk Sistem Pencarian Multimodal

Andi Muhammad Adlyn F. K. P.
Fakultas Ilmu Komputer
Universitas Indonesia
Depok, Indonesia
andi.muhammad37@ui.ac.id

Argya Farel Kasyara
Fakultas Ilmu Komputer
Universitas Indonesia
Jakarta Selatan, Indonesia
argya.farel@ui.ac.id

Vissuta Gunawan Lim
Fakultas Ilmu Komputer
Universitas Indonesia
Jakarta Barat, Indonesia
vissuta.gunawan@ui.ac.id

Abstract—Penelitian ini berfokus pada pengembangan search engine e-commerce yang efektif dan optimal untuk meningkatkan relevansi pencarian produk. Studi ini mengintegrasikan teknik Pemrosesan Bahasa Alami (NLP) dan Temu Balik Informasi (*Information Retrieval*), mengimplementasikan *dense retrieval*. Secara spesifik, penelitian ini memanfaatkan pendekatan *bi-encoder* (*sentence-transformers/all-MiniLM-L6-v2*) dan *cross-encoder* (*cross-encoder/ms-marco-MiniLM-L-12-v2*) yang dioptimalkan melalui pembelajaran kontrastif (*contrastive learning*). Dataset WANDS (*Wayfair Article Neural Dataset for Search*) digunakan untuk pelatihan dan evaluasi. Penelitian ini menerapkan berbagai teknik pemrosesan data, termasuk penanganan *missing values*, *lowercasing*, *lemmatization*, dan penghapusan *stopwords*. Performa model terutama dievaluasi menggunakan metrik *Normalized Discounted Cumulative Gain* (NDCG). Hasil menunjukkan bahwa *cross-encoder* yang telah di-*fine-tune* mengungguli model-model lainnya, termasuk model *pre-trained* yang lebih besar, membuktikan efektivitas *fine-tuning* yang spesifik untuk aplikasi pencarian e-commerce. Studi ini berkontribusi pada kemajuan teknologi pencarian e-commerce, memberikan

wawasan tentang keseimbangan antara kompleksitas model dan performa untuk implementasi praktis.

Keywords—*artificial intelligence, deep learning, information retrieval, language model, machine learning, natural language processing, search engine, semantic search, transformers*

I. PENDAHULUAN

A. Latar Belakang

Dalam beberapa dekade terakhir, *e-commerce* mengalami pertumbuhan signifikan yang menjadikannya salah satu sektor yang paling dinamis dan menguntungkan secara global. [1] Berdasarkan data dari Shopify, penjualan ritel *e-commerce* global diperkirakan melebihi 6 triliun dolar AS di 2024 dan diproyeksikan akan terus bertumbuh di tahun-tahun mendatang. Pertumbuhan tersebut didorong oleh peningkatan kemudahan akses internet, kemajuan teknologi, dan peralihan pola perilaku konsumen dari belanja konvensional ke belanja online.

Peralihan perilaku konsumen tersebut disebabkan oleh kemudahan dan kenyamanan yang ditawarkan perusahaan *e-commerce* yang memungkinkan konsumen untuk melakukan pembelian produk dari berbagai kategori dari mana saja dan kapan saja. Tentunya hal tersebut mendesak perusahaan *e-commerce* untuk meningkatkan ketersediaan produk yang

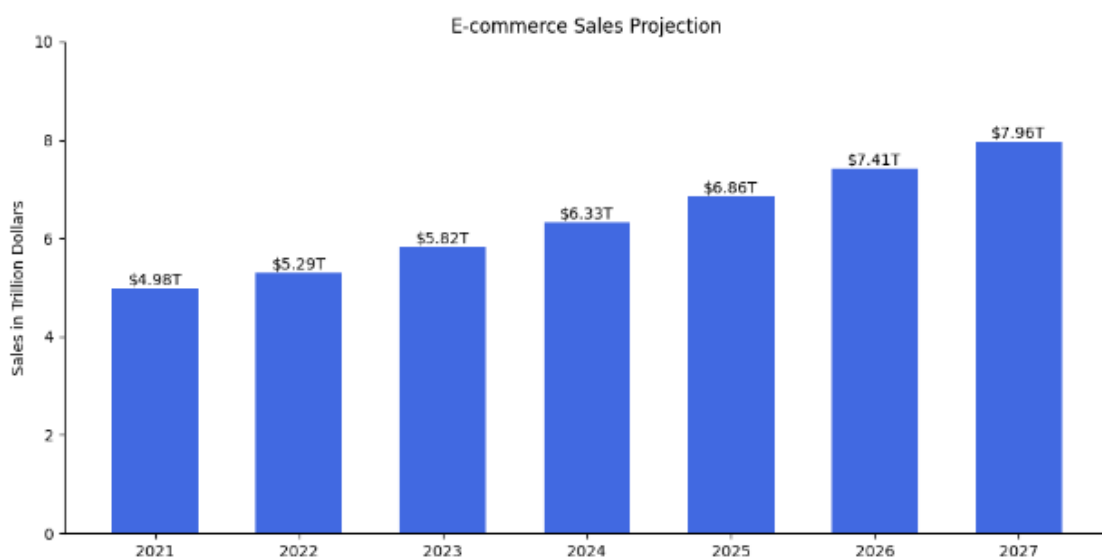


Figure 1. Proyeksi Penjualan E-commerce Global

dapat diperjualbelikan secara global. Desakan tersebut membawa tantangan baru bagi perusahaan *e-commerce*, yaitu menyajikan pengalaman pencarian produk yang optimal dan relevan sesuai dengan pencarian konsumen.

Dalam menghadapi tantangan penyajian pengalaman pencarian produk yang optimal, perusahaan *e-commerce* perlu mengembangkan sistem pencarian yang canggih dan efisien. Sistem ini harus mampu memahami intensi pengguna, mencocokkannya dengan katalog produk yang besar dan beragam, serta menampilkan hasil yang paling relevan dalam waktu singkat. Pemanfaatan teknologi kecerdasan buatan (AI) dan pembelajaran mesin (*machine learning*) menjadi krusial dalam upaya ini. Teknik-teknik seperti pemrosesan bahasa alami (NLP) dan sistem temu balik informasi (*information retrieval*) memungkinkan pemahaman yang lebih baik terhadap kueri pengguna dan peningkatan akurasi hasil pencarian. Namun, implementasi teknologi ini juga membawa tantangan tersendiri, seperti kebutuhan akan data *training* yang besar dan kompleksitas komputasi. Oleh karena itu, pengembangan sistem pencarian produk yang efektif dan adaptif menjadi faktor kunci dalam mempertahankan daya saing dan meningkatkan kepuasan pelanggan di industri *e-commerce* yang semakin kompetitif.

B. Tujuan

Tujuan utama dari penelitian ini adalah mengembangkan *search engine e-commerce* yang efektif dan optimal untuk meningkatkan relevansi hasil pencarian produk. Secara spesifik, penelitian ini bertujuan untuk: (1) mengintegrasikan disiplin ilmu Pemrosesan Bahasa Alami (NLP) dan Temu Balik Informasi (*Information Retrieval*) dalam pemeringkatan hasil pencarian, (2) mengimplementasikan metode *dense retrieval*, (3) memanfaatkan kemampuan *cross-encoder* dalam memahami hubungan semantik antara pasangan kueri-produk sehingga dapat melakukan pemeringkatan (4) mengoptimalkan model *cross-encoder* menggunakan teknik pembelajaran kontrastif (*contrastive learning*), dan (5) mengevaluasi efektivitas sistem yang dikembangkan menggunakan metrik standar dalam *information retrieval*, khususnya sistem pemeringkatan, seperti NDCG. Melalui pencapaian tujuan-tujuan ini, penelitian diharapkan dapat menghasilkan solusi pencarian yang tidak hanya meningkatkan akurasi tetapi juga memperkaya pengalaman pengguna dalam menemukan produk yang relevan di platform *e-commerce*.

C. Manfaat

Manfaat utama dari penelitian ini adalah memberikan solusi komprehensif atas tantangan pencarian produk yang dihadapi perusahaan *e-commerce*. Dengan mengintegrasikan NLP dan *information retrieval*, penelitian ini bertujuan untuk meningkatkan relevansi dan efektivitas hasil pencarian yang secara langsung berdampak pada peningkatan pengalaman pengguna. Secara operasional, sistem ini dapat mengoptimalkan efisiensi platform *e-commerce* dengan mengurangi beban pada tim dukungan pelanggan dan mempercepat proses pencarian produk. Dari perspektif bisnis, penelitian ini mendukung adaptabilitas platform terhadap perubahan tren pasar dan perilaku konsumen sehingga meningkatkan daya saing jangka panjang. Selain

itu, kontribusi ilmiah dari penelitian ini dalam bidang NLP dan *information retrieval* dapat menjadi landasan untuk inovasi lebih lanjut dalam teknologi *e-commerce*, membuka peluang untuk pengembangan dan penelitian di masa depan.

II. METODOLOGI

A. Pengumpulan Data

Dataset yang digunakan untuk penelitian ini adalah WANDS (*Wayfair Article Neural Dataset for Search*), yang dirilis pada bulan Januari tahun 2022 oleh Wayfair, sebuah perusahaan *e-commerce* yang berbasis di kota Boston, Massachusetts, Amerika Serikat. [2] Dataset ini dirancang khusus untuk mengevaluasi model *retrieval* untuk pencarian produk pada perusahaan *e-commerce* dan mengukur relevansi hasil pencarian dengan kueri pengguna. Keselarasan antara tujuan dataset ini dengan tujuan penelitian menjadikannya pilihan yang ideal untuk penelitian ini.

B. Analisis Dataset

WANDS (*Wayfair Article Neural Dataset for Search*) terdiri dari 3 dokumen berformat csv. Berikut adalah penjelasan dari setiap dokumen beserta kolom penyusunnya:

1. query.csv:

- Menyimpan kueri pencarian pengguna
- Jumlah baris: 480
- Jumlah kolom: 3
- Nama kolom:
 - *query_id* : Identifikator unik untuk setiap kueri (0 missing values)
 - *query* : String kueri (0 missing values)
 - *query_class* : Kategori kueri (6 missing values)

2. product.csv:

- Menyimpan semua produk kandidat
- Jumlah baris: 42994
- Jumlah kolom: 9
- Nama kolom:
 - *product_id* : Identifikator unik untuk setiap produk (0 missing values)
 - *product_name* : String kueri (0 missing values)
 - *product_class* : Kategori produk (2852 missing values)
 - *category_hierarchy* : Kategori induk produk, dipisahkan oleh '/' (1556 missing values)
 - *product_description* : String deskripsi produk (6008 missing values)
 - *product_features* : Atribut produk, yaitu nilai yang mendeskripsikan produk, dipisahkan oleh '/' (0 missing values)

- *rating_count* : Jumlah penilaian pengguna (9452 *missing values*)
- *average_rating* : Rerata penilaian pengguna yang diterima (9452 *missing values*)
- *review_count* : Jumlah ulasan pengguna (9452 *missing values*)

3. label.csv:

- Menyimpan pasangan (produk, relevansi) yang telah dianotasi
- Jumlah baris: 233448
- Jumlah kolom: 4
- Nama kolom:
 - *id* : Identifikator unik untuk setiap anotasi
 - *query_id* : Identifikator kueri
 - *product_id* : Identifikator produk
 - *label* : Label relevansi (*Exact, Partial, Irrelevant*)

Berdasarkan informasi dataset, dapat ditinjau bahwa pada label.csv, terdapat kolom yang beririsan dengan dokumen csv lain, yaitu *query_id* pada query.csv dan *product_id* pada product.csv. Dengan begitu, dapat dilakukan *inner join* antara label.csv dengan query.csv pada kolom *query_id*. Hasilnya kemudian akan dilakukan *inner join* dengan product.csv pada kolom *product_id*.

Setelah melakukan *joins* tersebut diperoleh sebuah *dataframe* yang terdiri dari 233448 kolom dan 14 kolom, mencakup semua informasi dari ketiga dokumen. Jumlah baris *dataframe* setelah *join* sama jumlahnya dengan jumlah baris pada label.csv. Hal tersebut mengindikasikan bahwa tidak ada kehilangan data akibat *data mismatch* pada proses *join*. Berikut adalah rincian *missing values* pada setiap kolom setelah melakukan *join*:

- *query_class* : 328 *missing values*
- *product_hierarchy* : 17963 *missing values*
- *category_hierarchy* : 7071 *missing values*
- *product_description* : 30784 *missing values*
- *rating_count* : 52718 *missing values*
- *average_rating* : 52718 *missing values*
- *review_count* : 52718 *missing values*

C. Pemrosesan Data

Pemrosesan data adalah sebuah langkah krusial dalam pengembangan sistem pencarian produk yang optimal. Proses ini dilakukan untuk membersihkan data dan menormalisasi data teks yang berpotensi meningkatkan performa sistem pencarian.

Adapun langkah-langkah pemrosesan data yang dilakukan adalah sebagai berikut:

1. Penanganan *missing values*

Pendekatan yang diambil untuk menangani *missing values* adalah *data imputation*. Untuk kolom dengan data tekstual seperti *query_class*, *product_hierarchy*, *category_hierarchy*, dan *product_description*, *missing values* diisi dengan string kosong (''), sedangkan untuk kolom dengan data numerik seperti *rating_count*,

average_rating, dan *review_count*, *missing values* diisi dengan median dari masing-masing kolom. Pendekatan ini dipilih dibandingkan *row-wise deleting* untuk menghindari kehilangan informasi yang disebabkan oleh penghapusan sampel yang mengandung *missing value(s)*.

2. Lowercasing

Lowercasing adalah proses konversi seluruh teks pada dataset menjadi huruf kecil. Langkah ini diambil untuk memastikan konsistensi kapitalisasi dari seluruh data teks pada dataset sehingga dapat menghindari duplikasi data teks akibat perbedaan kapitalisasi.

3. Lemmatization

[3] *Lemmatization* adalah proses menghilangkan atau mengganti awalan maupun akhiran yang infleksional dari sebuah kata dan mengembalikan bentuk dasar atau bentuk kamus (*lemma*) dari kata tersebut untuk mengurangi variasi kata. [4] Selain mereduksi kata menjadi bentuk dasarnya, *lemmatization* juga menghubungkan kata dengan makna yang serupa menjadi satu kata serta memahami konteks kalimat dan *part of speech*. Pemahaman akan konteks ini yang membedakan *lemmatization* dan *stemming* yang hanya "memotong" awalan atau akhiran kata dalam menghasilkan bentuk dasar sebuah kata. Contoh *lemmatization*: "*cooking*" menjadi "*cook*", "*better*" menjadi "*good*", dan "*leaves*" menjadi "*leaf*". Dalam konteks sistem pencarian produk *e-commerce*, *lemmatization* memainkan peran krusial dalam meningkatkan akurasi dan relevansi hasil pencarian. Proses ini membantu sistem memahami bahwa kata-kata seperti "*chairs*", "*chairing*", dan "*chaired*" merujuk pada kata yang sama dengan "*chair*" sehingga memungkinkan pencocokan yang lebih akurat antara kueri pengguna dan deskripsi produk.

4. Penghapusan *stopwords*

Penghapusan *stopwords* adalah proses menghilangkan kata-kata yang sangat umum dan tidak membawa makna spesifik dalam konteks pencarian atau analisis teks. *Stopwords* umumnya meliputi kata-kata seperti "*and*", "*or*", "*the*", "*is*", dan kata-kata fungsi lainnya yang memiliki frekuensi tinggi namun kontribusi semantik rendah. Proses ini penting dalam pengolahan bahasa alami dan pencarian informasi karena dapat mengurangi *noise* dalam data, meningkatkan efisiensi komputasi, dan memfokuskan analisis pada kata-kata yang lebih bermakna. Dalam konteks sistem pencarian produk *e-commerce*, penghapusan *stopwords* dapat membantu meningkatkan relevansi hasil pencarian dengan menghilangkan kata-kata yang tidak berkontribusi signifikan terhadap makna kueri atau deskripsi produk.

5. Pembuatan *Text Embeddings*

[5] *Text embeddings* adalah representasi vektor dari bahasa alami yang mengkodekan informasi semantiknya. *Text embeddings* banyak digunakan dalam berbagai tugas pemrosesan bahasa alami (NLP), seperti temu balik informasi (*Information Retrieval*), sistem tanya jawab, kesamaan tekstual semantik, penambangan *bitext*, rekomendasi *item*, dan lain-lain. Dalam bidang temu balik

informasi (*Information Retrieval*), tahap pertama pencarian sering bergantung pada *text embeddings* untuk secara efisien menemukan sejumlah kecil dokumen kandidat dari korpus skala besar menggunakan teknik pencarian tetangga terdekat aproksimasi.

Pembuatan *text embeddings* umumnya dilakukan menggunakan *encoder* yang merupakan salah satu bagian dari arsitektur *transformer*, seperti BERT, RoBERTa, atau Sentence-BERT. Dalam bidang *information retrieval* dan NLP, *bi-encoder*, *cross-encoder*, dan *transformer encoder* adalah model yang paling sering digunakan untuk membuat *text embeddings*.

Pada penelitian ini, jenis *encoder* yang digunakan adalah *bi-encoder*, [6] jenis *encoder* yang meng-*encode* setiap *input* secara independen sebelum menggabungkan *embedding* yang dihasilkan untuk menghitung skor kesamaan, dan *cross-encoder*, jenis *encoder* yang memproses pasangan *input* dan mempertimbangkan interaksi antar *input* yang dalam kasus ini adalah pasangan kueri dan teks produk. Secara spesifik, *bi-encoder* yang digunakan adalah *sentence-transformers/all-MiniLM-L6-v2*, sedangkan *cross-encoder* yang digunakan adalah *cross-encoder/ms-marco-MiniLM-L-12-v2*.

Model *bi-encoder all-MiniLM-L6-v2* merupakan varian dari arsitektur MiniLM yang terdiri dari 6 *layer transformer* dengan ukuran *hidden state* 384 dimensi. Model ini menghasilkan *embedding* vektor dengan dimensi 384 untuk setiap *input* teks. Konfigurasi yang lebih ringkas ini memungkinkan proses *encoding* yang lebih cepat dan efisien, sangat cocok untuk tahap pencarian awal dalam sistem *information retrieval* skala besar.

Sementara itu, *cross-encoder ms-marco-MiniLM-L-12-v2* menggunakan arsitektur MiniLM dengan 12 *layer*, memiliki ukuran *hidden state* 384 dimensi, dan 12 *attention head*. Model ini tidak menghasilkan *embedding* terpisah, melainkan skor relevansi langsung untuk pasangan *input*. Dengan jumlah *layer* yang lebih banyak, model ini dapat menangkap interaksi yang lebih kompleks antara kueri dan teks produk, membuatnya ideal untuk tahap perangkingan yang lebih presisi.

Alasan pemilihan *encoder* tersebut adalah [7] performa MiniLM yang cukup baik saat dilakukan 7 dari 9 asesmen GLUE (*General Language Understanding Evaluation*) yang mencakup sembilan *sentence-level classification tasks*, termasuk *Multi-Genre Natural Language Inference (MNLI)* (Williams et al., 2018), *Stanford Sentiment Treebank (SST)* (Socher et al., 2013), *Question Natural Language Inference (QNLI)* (Rajpurkar et al., 2016), *Corpus of Linguistic Acceptability (CoLA)* (Warstadt et al., 2018), *Recognizing Textual Entailment*

(*RTE*) (Dagan et al., 2006; Bar-Haim et al., 2006; Giampiccolo et al., 2007; Bentivogli et al., 2009), *Microsoft Research Paraphrase Corpus (MRPC)* (Dolan & Brockett, 2005), *Quora Question Pairs (QQP)* (Chen et al., 2018), serta asesmen lain seperti *Stanford Question Answering Dataset (SQuAD2)*. Metrik evaluasi yang digunakan untuk setiap dataset adalah akurasi, kecuali SQuAD2 yang menggunakan skor F1.

Pada Figure 2, dapat dilihat bahwa MiniLM menunjukkan performa yang kompetitif dibandingkan dengan model-model lainnya. MiniLM menunjukkan performa yang sangat baik dalam beberapa tugas spesifik, seperti SST-2, QNLI, dan QQP dengan akurasi 92.0, 91.0, dan 91.0 secara berurutan yang hampir menyamai BERT BASE. Dengan jumlah parameter yang sama dengan DistilBERT dan TinyBERT (66M), MiniLM berhasil mencapai skor rata-rata 80.4, yang hanya sedikit di bawah BERT BASE (81.5) yang memiliki parameter hampir dua kali lipat (109M). Keunggulan MiniLM terlihat jelas jika dibandingkan dengan DistilBERT dan TinyBERT, di mana MiniLM konsisten mengungguli keduanya di hampir semua tugas. Hal ini menunjukkan bahwa MiniLM mampu mempertahankan kapabilitas pemahaman bahasa yang kuat sambil tetap efisien dalam hal ukuran model, menjadikannya pilihan yang menjanjikan untuk aplikasi NLP dan *information retrieval* yang membutuhkan keseimbangan antara performa dan efisiensi komputasional.

6. Label Encoding

Label relevansi dalam WANDS terbagi menjadi tiga kategori, yaitu *Exact*, *Partial*, dan *Irrelevant*. Tahap *label encoding* dilakukan karena model pembelajaran mesin tidak bisa *input* nonnumerik. Untuk tugas pemeringkatan hasil pencarian produk, label ini di-*encode* dengan pemetaan sebagai berikut: *Exact*: 1, *Partial*: 0.5, *Irrelevant*: 0. *Encoding* ini dipilih karena mencerminkan tingkat relevansi secara ordinal dan kompatibel dengan metrik evaluasi seperti NDCG. Pemilihan nilai 1, 0.5, dan 0 tidak hanya merefleksikan urutan tingkat relevansi, tetapi juga berkaitan erat dengan perhitungan *gain* dalam NDCG. Nilai *gain* yang lebih tinggi untuk label *Exact* (1) dibandingkan *Partial* (0.5) memastikan bahwa hasil pencarian yang sangat relevan mendapatkan bobot yang lebih besar dalam perhitungan NDCG. Selain itu, penggunaan skala 0—1 ini memudahkan proses normalisasi dalam NDCG, di mana skor akhir dibagi dengan skor ideal, menghasilkan nilai antara 0 dan 1 yang mudah diinterpretasikan dan dibandingkan antar kueri yang berbeda.

Model	#Param	SQuAD2	MNLI-m	SST-2	QNLI	CoLA	RTE	MRPC	QQP	Average
BERT _{BASE}	109M	76.8	84.5	93.2	91.7	58.9	68.6	87.3	91.3	81.5
DistilBERT	66M	70.7	79.0	90.7	85.3	43.6	59.9	87.5	84.9	75.2
TinyBERT	66M	73.1	83.5	91.6	90.5	42.8	72.2	88.4	90.6	79.1
MINILM	66M	76.4	84.0	92.0	91.0	49.2	71.5	88.4	91.0	80.4

Figure 2. Performa berbagai encoder pada benchmark GLUE dan SQuAD2.

D. Pemodelan

Terdapat 3 konfigurasi model yang akan dievaluasi performanya dalam pemeringkatan hasil pencarian produk, di antaranya:

1. BM25 Sparse Retrieval

[8] *Okapi BM25* atau yang biasa disebut dengan BM25 (*Best Matching 25*) adalah suatu sistem pemeringkatan yang digunakan untuk mengurutkan hasil kecocokan terhadap dokumen-dokumen, berdasarkan kata kunci yang dicarinya, dalam kasus ini adalah kueri. BM25 merupakan formula terbaik dalam kelas *best match* karena formula ini efektif dan memiliki ketepatan dalam mengurutkan dokumen berdasarkan kueri yang dicari. Namun, kekurangan dari BM25 adalah ketidakmampuan menangkap makna semantik dari serangkaian kata dan hanya mengandalkan *exact keyword match*. [9] Misal terdapat kueri Q yang mengandung kata kunci $q_1, q_2, \dots, q_n$, nilai BM25 dari dokumen D , yang dalam kasus ini adalah teks produk, adalah:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \times \frac{f(q_i, D) \times (k_1 + 1)}{f(q_i, D) + k_1 \times \left(1 - b + b \times \frac{|D|}{\text{avgdl}}\right)}$$

di mana $f(q_i, D)$ adalah *term frequency* dari q_i pada dokumen D , $|D|$ adalah jumlah kata pada dokumen D , dan avgdl merupakan rata-rata panjang dokumen yang dihitung dari seluruh dokumen dalam koleksi teks yang digunakan. k_1 dan b adalah parameter bebas yang biasanya dipilih dan jika tidak ada optimisasi lanjutan, $k_1 \in [1.2, 2.0]$ dan $b = 0.75$. $\text{IDF}(q_i)$ adalah bobot IDF (*Inverse Document Frequency*) dari kata kunci/kueri q_i . Biasanya dihitung sebagai:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

di mana N adalah jumlah total dokumen dalam koleksi, dan $n(q_i)$ adalah jumlah dokumen yang mengandung q_i . Produk kandidat sebanyak k dengan skor BM25 terbesar yang akan dikembalikan.

2. Bi-Encoder Dense Retrieval

Konfigurasi ini menggunakan *bi-encoder sentence-transformers/all-MiniLM-L6-v2* untuk melakukan *dense retrieval*. Proses dimulai dengan meng-*encode* kueri (Q) dan setiap dokumen produk (D_i) secara independen menggunakan persamaan $H_Q = T(Q)$ dan $H_{D_i} = T(D_i)$, di mana T adalah fungsi *encoding* dari *bi-encoder*. Selanjutnya, kesamaan antara kueri-produk dihitung menggunakan *cosine similarity*. [10] *Cosine similarity* merupakan pengukuran matematis yang digunakan untuk mengukur kemiripan dua vektor dalam ruang multidimensi, khususnya dalam ruang berdimensi tinggi, dengan menghitung sudut kosinus di antara kedua vektor tersebut, yang dalam kasus ini adalah vektor kueri dan vektor teks produk, dengan persamaan $\text{sim} = \cos(H_Q, H_{D_i})$. Semakin tinggi nilai *cosine similarity* (mendekati 1), semakin mirip atau relevan kedua vektor tersebut, yang mengindikasikan kecocokan yang lebih tinggi antara kueri dan produk. Produk kandidat kemudian diurutkan berdasarkan *cosine similarity* untuk menghasilkan peringkat akhir.

3. Cross-Encoder

Konfigurasi ini menggunakan cross-encoder *cross-encoder/ms-marco-MiniLM-L-12-v2* untuk memproses pasangan kueri dan teks produk. [11] Prosesnya adalah sebagai berikut:

- Kueri (Q) dan teks produk (P) diproses melalui *cross-encoder* yang telah di-*pretrain* untuk menghasilkan *embedding* yang merepresentasikan similaritas antara kueri dan teks produk.
- Embedding* ini digunakan untuk melatih *BERTSequenceClassifier* yang merupakan *builtin classifier* dari *cross-encoder/ms-marco-MiniLM-L-12-v2* atau model pembelajaran mesin lainnya yang menghasilkan skor relevansi antara kueri dan produk.
- Produk-produk diurutkan berdasarkan skor relevansi untuk menghasilkan peringkat akhir.

Pendekatan ini memanfaatkan kemampuan *cross-encoder* dalam memahami interaksi antara kueri dan teks produk untuk meningkatkan akurasi pemeringkatan.

Terlebih lagi, bisa dilakukan *fine-tuning* untuk memperkaya konteks dengan dataset *e-commerce*. Proses *fine-tuning* dilakukan menggunakan pendekatan pembelajaran kontrastif (*contrastive learning*) sebanyak 1 *epoch* dan 4 *epoch* untuk meningkatkan performa model dalam tugas pemeringkatan produk.

Pembelajaran kontrastif (*contrastive learning*) adalah sebuah paradigma pembelajaran yang bertujuan untuk membuat representasi vektor dari pasangan kueri-produk yang serupa menjadi lebih dekat dalam ruang vektor, sementara membuat representasi vektor dari pasangan kueri-produk yang berbeda menjadi lebih jauh dalam ruang vektor. Dalam konteks pencarian produk *e-commerce*, pendekatan ini melatih model untuk menghasilkan *embedding* yang mirip untuk pasangan kueri-produk yang relevan, dan *embedding* yang berbeda untuk pasangan yang tidak relevan.

Adapun langkah *fine tuning* dengan pembelajaran kontrastif adalah sebagai berikut: (1) Dataset dibagi menjadi set pelatihan dan pengembangan dengan setiap sampel pelatihan terdiri dari pasangan kueri-produk dan label relevansi. (2) Model dilatih menggunakan *DataLoader* dengan *loss function* MSE (yang dipilih karena kemampuannya dalam mengoptimalkan jarak relatif antara *embedding* kueri dan produk dalam ruang vektor sesuai dengan prinsip pembelajaran kontrastif, sambil tetap mempertahankan interpretabilitas skor relevansi) dan dievaluasi secara berkala menggunakan *CERerankingEvaluator*. (3) Setelah *fine-tuning*, kueri (Q) dan teks produk (P) diproses melalui *cross-encoder* yang telah dioptimalkan untuk menghasilkan skor

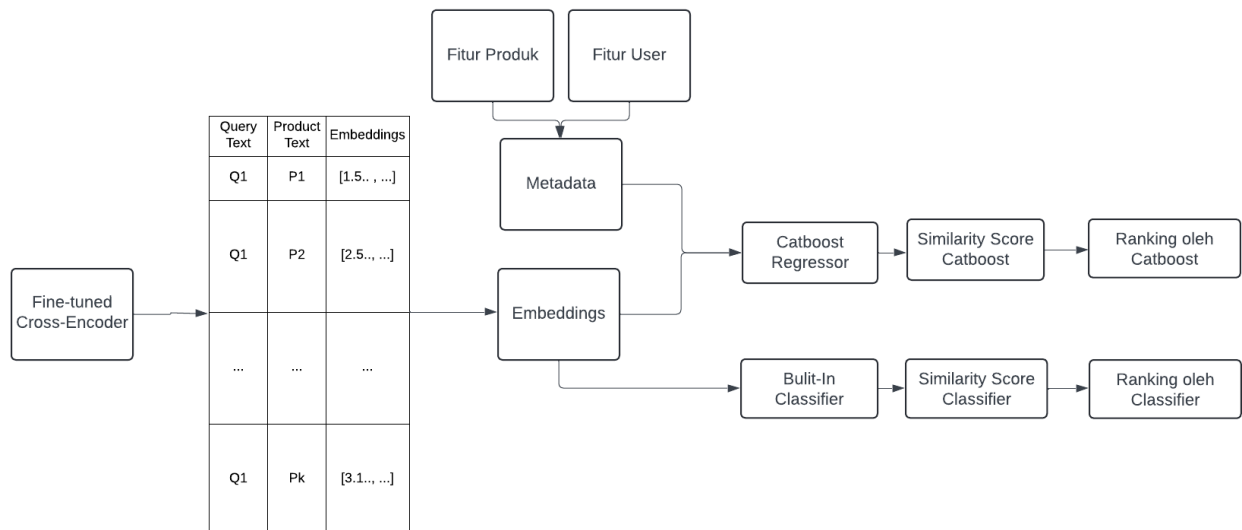


Figure 3. Skema pemeringkatan ulang untuk kueri teks dengan cross-encoder sebagai query-product embedder dan Catboost Regressor serta Built-In Classifier sebagai penghasil similarity score. (Penggunaan Fitur User di luar cakupan penelitian ini)

relevansi, yang kemudian digunakan langsung untuk mengurutkan produk-produk berdasarkan kesamaannya dengan kueri. Pendekatan ini memanfaatkan kemampuan *cross-encoder* yang telah ditingkatkan untuk lebih memahami konteks spesifik dari dataset produk *e-commerce* sehingga dapat menghasilkan pemeringkatan yang lebih akurat dan relevan.

4. Pemeringkatan berdasarkan skor kesamaan dari *BAAI/bge-reranker-large*
BAAI/bge-reranker-large adalah sebuah *cross-encoder* yang memiliki parameter sebanyak 560M. *bge-reranker-large* melakukan proses yang sama dengan *cross-encoder*. Adapun penggunaan model ini hanya sebagai perbandingan terhadap model lainnya.

5. *CLIP (Contrastive Language-Image Pre-Training)*
[12] *CLIP (Contrastive Language-Image Pre-Training)* adalah sebuah *neural network* yang dilatih pada kumpulan pasangan gambar dan teks. Proses pelatihan ini membuat *CLIP* dapat memetakan gambar dan teks ke ruang vektor dengan dimensi yang sama. *CLIP* dapat diberikan instruksi dalam bahasa alami untuk memprediksi potongan teks yang paling relevan berdasarkan gambar yang diberikan, tanpa perlu optimisasi langsung untuk tugas tersebut.
Pada penelitian ini, *CLIP* digunakan untuk implementasi *visual search* sebagai *text embedder* sekaligus sebagai fitur tambahan, yaitu *visual search*.

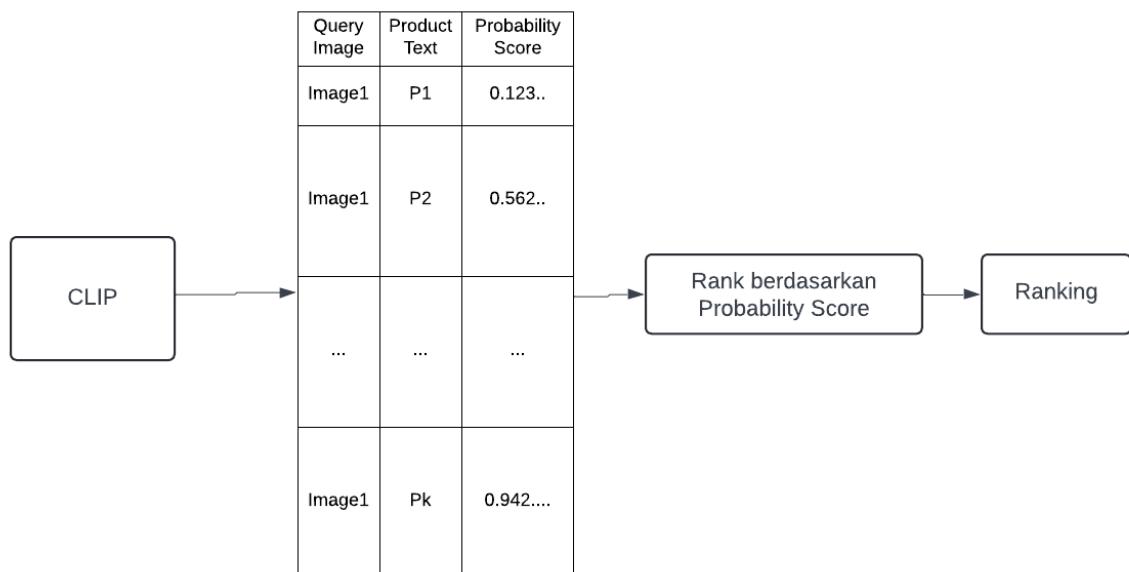


Figure 4. Skema pemeringkatan ulang untuk kueri gambar dengan CLIP.

III. HASIL DAN PEMBAHASAN

Konfigurasi	Metrik	NDCG@10	NDCG@20	NDCG@all
Pemeringkatan berdasarkan <i>random similarity</i>		0.572	0.623	0.881
Pemeringkatan berdasarkan skor BM25		0.774	0.787	0.935
Pemeringkatan berdasarkan <i>cosine similarity</i> dari <i>Bi-Encoder Dense Retrieval</i> dengan CLIP sebagai <i>text embedder</i>		0.719	0.735	0.922
Pemeringkatan berdasarkan <i>cosine similarity</i> dari <i>Bi-Encoder Dense Retrieval</i> dengan <i>sentence-transformers/all-MiniLM-L6-v2</i> sebagai <i>text embedder</i>		0.816	0.831	0.948
Pemeringkatan berdasarkan skor relevansi dari <i>Cross-Encoder</i>		0.827	0.833	0.944
Pemeringkatan berdasarkan skor relevansi dari <i>BAAI/bge-reranker-large</i> sebagai <i>ranking model</i>		0.833	0.835	0.945
Pemeringkatan berdasarkan embedding dari <i>Cross-Encoder (fine-tuned 4 epoch)</i> dengan <i>CatBoostRegressor</i> sebagai <i>ranking model</i>		0.832	0.841	0.951
Pemeringkatan berdasarkan embedding dari <i>Cross-Encoder (fine-tuned 4 epoch)</i> dan <i>metadata</i> dengan <i>CatBoostRegressor</i> sebagai <i>ranking model</i>		0.836	0.848	0.952
Pemeringkatan berdasarkan skor relevansi dari <i>Cross-Encoder</i> yang telah dilakukan <i>fine tune</i> sebanyak 1 <i>epoch</i>		0.847	0.847	0.956
Pemeringkatan berdasarkan skor relevansi dari <i>Cross-Encoder</i> yang telah dilakukan <i>fine tune</i> sebanyak 4 <i>epoch</i>		0.850	0.857	0.957

*Catatan: Evaluasi model CLIP tidak bisa dilakukan karena keterbatasan data untuk memvalidasi performa model dan daya komputasi, hasil implementasi *visual search* bisa dilihat pada lampiran.

[13] Dalam konteks pemeringkatan pada sistem pencarian produk, evaluasi yang dilakukan tentunya berbeda dengan klasifikasi dan regresi yang menggunakan pengukuran performa sederhana seperti MSE, akurasi dan metrik evaluasi lainnya. Misal terdapat n objek yang akan diberi peringkat, metrik evaluasi pemeringkatan harus mampu menginduksi urutan total pada $n!$ kemungkinan hasil peringkat. Total kemungkinan urutan ini dapat menjadi sangat besar bahkan untuk nilai n yang relatif kecil.

Kompleksitas ini menyebabkan ketiadaan pengukuran pemeringkatan optimal tunggal yang berlaku untuk semua aplikasi. Setiap metrik memiliki kelebihan dan kekurangannya masing-masing, tergantung pada konteks dan tujuan spesifik dari sistem pemeringkatan yang sedang dievaluasi. Hal ini menjadi tantangan tersendiri bagi peneliti.

Dalam upaya untuk mengatasi tantangan ini, beberapa metrik evaluasi telah dikembangkan secara luas dalam bidang *information retrieval* dan sistem pencarian produk. Salah satu metrik yang paling populer dan efektif adalah *Normalized Discounted Cumulative Gain* (NDCG@k). NDCG@k didasarkan pada dua prinsip utama:

1. Dokumen atau item yang sangat relevan lebih berharga daripada item yang sedikit relevan.
2. Semakin rendah posisi peringkat sebuah produk, semakin kecil nilainya pada persamaan.

Untuk memahami NDCG, perlu diketahui komponen penyusunnya, yaitu *Discounted Cumulative Gain* (DCG@k) dan *Ideal Discounted Cumulative Gain* (IDCG@k).

DCG@k mengukur kualitas pemeringkatan berdasarkan posisi setiap produk dalam daftar hasil sebanyak k produk dengan memberikan penalti pada item relevan yang muncul lebih rendah dalam daftar. Formula DCG@k didefinisikan sebagai berikut:

$$DCG@k(f, S_n) = \sum_{r=1}^k \frac{rel_r}{\log_2(1+r)}$$

Di mana f adalah fungsi pemeringkatan, S_n adalah dataset yang berisikan n produk kandidat, rel_r adalah nilai relevansi item pada posisi r , dan $\log_2(1+r)$ adalah faktor diskon yang mengurangi kontribusi item yang berada pada posisi lebih rendah.

IDCG@k adalah nilai DCG@k maksimum yang mungkin dicapai jika sistem menghasilkan pemeringkatan yang sempurna. IDCG@k dihitung dengan mengurutkan semua item berdasarkan relevansinya secara menurun dan kemudian menghitung DCG@k.

Akhirnya, NDCG@k diperoleh dengan membagi DCG@k aktual dengan IDCG@k sehingga nantinya mendapat hasil dengan rentang 0 – 1. Nilai NDCG@k 1 mengindikasikan bahwa pemeringkatan yang dihasilkan

berhasil memeringkatkan k produk kandidat secara 100% akurat dan nilai NDCG@k 0 sebaliknya. Berikut adalah persamaan NDCG@k:

$$NDCG@k(f, S_k) = \frac{DCG(f, S_k)}{IDCG(f, S_k)}$$

NDCG@k memiliki beberapa keunggulan sebagai metrik evaluasi pemeringkatan:

1. Memperhitungkan urutan peringkat item.
2. Menormalisasi skor sehingga dapat dibandingkan antara kueri yang berbeda.
3. Memberikan bobot lebih tinggi pada item yang sangat relevan.

Dengan menggunakan NDCG@k, kita dapat mengevaluasi kualitas hasil pemeringkatan sistem pencarian produk dengan lebih komprehensif dan objektif.

Berikut adalah beberapa nilai NDCG@k yang diperoleh selama penelitian dengan model-model yang telah disebutkan serta dengan k yang beragam (Table 1).

Berikut adalah analisis dan pembahasan dari hasil eksperimentasi yang diperoleh:

1. Pemeringkatan berdasarkan *random similarity* menghasilkan nilai NDCG@10 sebesar 0.572, NDCG@20 sebesar 0.623, dan NDCG@all sebesar 0.881. Hasil ini menjadi *baseline* terendah dalam eksperimen, yang sesuai dengan ekspektasi karena metode ini tidak mempertimbangkan relevansi sebenarnya antara kueri dan produk.
2. Pemeringkatan menggunakan skor BM25 menunjukkan peningkatan signifikan dengan NDCG@10 mencapai 0.774, NDCG@20 sebesar 0.787, dan NDCG@all 0.935. Hal ini membuktikan efektivitas BM25 dalam menangkap relevansi teks berdasarkan frekuensi kata dan panjang dokumen.
3. Penggunaan *bi-encoder* untuk *dense retrieval* menghasilkan performa yang lebih baik dibandingkan BM25, dengan NDCG@10 sebesar 0.816, NDCG@20 0.831, dan NDCG@all 0.948. Peningkatan ini menunjukkan keunggulan representasi *dense vector* dalam menangkap makna semantik dari kueri dan produk.
4. *Cross-encoder* tanpa *fine-tuning* menunjukkan performa yang lebih baik dari *bi-encoder* dengan NDCG@10 0.827, NDCG@20 0.833, dan NDCG@all 0.949. Ini menunjukkan keunggulan arsitektur *cross-attention* dalam memahami interaksi antara kueri dan teks produk dengan lebih baik.
5. Model *reranker* *CatBoostRegressor* yang menggunakan *embedding* dari *cross-encoder* (*fine-tuned 4 epoch*) dan *metadata* sebagai fitur prediktor menunjukkan skor yang lebih baik dibandingkan yang tidak menggunakan *metadata*. Peningkatan ini menunjukkan bahwa *metadata* memiliki peran dalam pemeringkatan produk. Meskipun demikian, model *reranker* ini menghasilkan skor yang lebih rendah dari *cross-encoder* yang menggunakan *built-in classifier*. Hal ini juga bisa disebabkan oleh *hyperparameter* model *reranker* yang belum di-tune.

6. Performa terbaik dicapai oleh *cross-encoder* yang telah di-*fine-tune*. Dengan 1 *epoch fine-tuning*, model mencapai NDCG@10 sebesar 0.847, NDCG@20 0.847, dan NDCG@all 0.956. Dengan 4 *epoch*, NDCG@10, NDCG@20, dan NDCG@all meningkat menjadi, berturut-turut, 0.850, 0.857, 0.957. Peningkatan ini menunjukkan efektivitas *fine-tuning* dalam mengadaptasi model *pre-trained* ke domain spesifik *e-commerce*, terutama dataset WANDS.
7. Menariknya, *cross-encoder* *MiniLM-L12-v2* (dengan 41M parameter) yang telah di-*fine-tune* mampu mengungguli *BAAI/bge-reranker-large* yang memiliki 560M parameter (NDCG@10: 0.850 vs 0.833). Hal ini menunjukkan bahwa ukuran model bukan satu-satunya faktor penentu performa. *Fine-tuning* yang tepat pada model yang lebih kecil dapat menghasilkan kinerja yang lebih baik dibandingkan dengan model yang jauh lebih besar.

IV. KESIMPULAN DAN SARAN

Dari penelitian yang telah dilakukan, dapat disimpulkan beberapa hal, yakni:

1. Penelitian ini berhasil mengembangkan dan mengevaluasi berbagai konfigurasi model untuk sistem pencarian produk *e-commerce*, mulai dari metode tradisional BM25 hingga pendekatan berbasis *deep learning* seperti *bi-encoder* dan *cross-encoder*.
2. Hasil eksperimen menunjukkan bahwa metode *dense retrieval*, khususnya yang menggunakan *bi-encoder*, secara konsisten mengungguli metode *sparse retrieval* seperti BM25 dalam hal akurasi pemeringkatan yang diukur menggunakan metrik NDCG.
3. Proses *fine-tuning* pada *cross-encoder* terbukti sangat efektif dalam meningkatkan performa model, dengan peningkatan NDCG yang signifikan setelah hanya beberapa *epoch* pelatihan.
4. *Cross-encoder* yang telah di-*fine-tune* (*MiniLM-L12-v2* dengan 41M parameter) mampu mengungguli model yang jauh lebih besar seperti *BAAI/bge-reranker-large* (560M parameter), menunjukkan bahwa efisiensi komputasi dapat dicapai tanpa mengorbankan kualitas hasil.
5. Keterlibatan *metadata* dalam proses *reranking* meningkatkan performa sistem pencarian. Selain itu, pemilihan model *reranking* dan proses training model memainkan peran yang sama pentingnya.

Berikut adalah beberapa hal yang bisa dilakukan untuk meningkatkan kualitas penelitian:

1. Eksplorasi Arsitektur Model: Penelitian lebih lanjut dapat dilakukan untuk mengeksplorasi arsitektur model lain atau kombinasi model yang mungkin lebih efektif untuk tugas pencarian produk *e-commerce*.
2. Optimisasi *Hyperparameter*: Studi lebih mendalam tentang pengaruh berbagai *hyperparameter*, terutama dalam proses *fine-tuning*, dapat dilakukan untuk mengoptimalkan performa model.

3. Analisis Kualitatif: Selain evaluasi kuantitatif menggunakan NDCG, analisis kualitatif terhadap hasil pencarian dapat memberikan wawasan lebih mendalam tentang kekuatan dan kelemahan masing-masing pendekatan.
4. Skalabilitas dan Efisiensi: Penelitian lebih lanjut pada sistem secara menyeluruh (Figure 5) diperlukan untuk mengevaluasi skalabilitas dan efisiensi komputasi dari model-model yang diusulkan dalam skenario dunia nyata dengan dataset yang lebih besar.
7. Evaluasi Lintas Domain: Menguji model pada dataset *e-commerce* dari domain atau bahasa yang berbeda dapat memberikan wawasan tentang generalisasi dan adaptabilitas model.

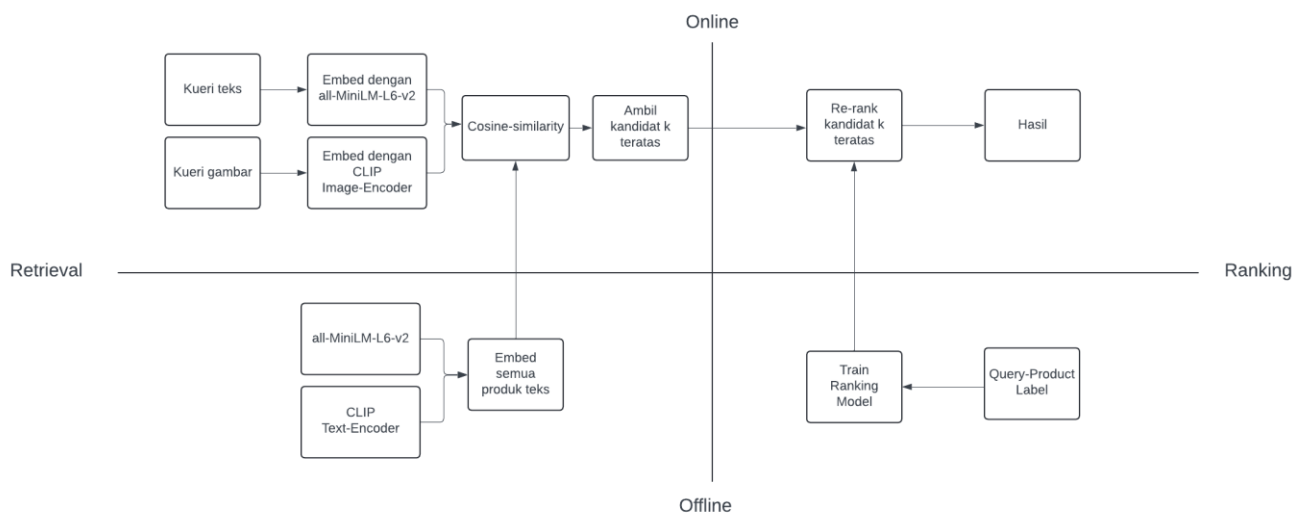


Figure 5. Skema sistem pencarian.

Figure 5 menggambarkan skema sistem pencarian secara menyeluruh, skema terbagi menjadi empat bagian:

- a. (Kiri-Bawah), *Product-embedding* dan pembuatan *index*.
- b. (Kanan-Bawah), Proses *Training ranking model*.
- c. (Kiri-Atas), Proses *Retrieval*, menerima dan meng-embed input (baik teks atau gambar). Kemudian dilakukan retrieval berdasarkan *cosine similarity query embedding* dan *product embedding* yang bersesuaian.
- d. (Kanan-Atas), Proses *Ranking*, produk – produk yang diperoleh dari proses 3 dilakukan pemeringkatan ulang menggunakan *ranking model* yang sudah di-*train*.
5. Integrasi Fitur Multimodal: Mengingat pentingnya aspek visual dalam *e-commerce*, integrasi pencarian berbasis gambar (seperti yang diimplementasikan menggunakan CLIP) dengan pencarian berbasis teks dapat dieksplorasi lebih lanjut.
6. Personalisasi: Pengembangan model yang dapat mengintegrasikan preferensi pengguna dan riwayat pencarian untuk hasil yang lebih personal dapat menjadi arah penelitian yang menjanjikan.

ACKNOWLEDGMENT

Penelitian ini tidak dapat dilaksanakan tanpa ketersediaan dataset WANDS dari Wayfair.

REFERENCES

- [1] G. Bernhardt, "Global Ecommerce Sales (2020–2025)," *Shopify*, Apr. 26, 2022. <https://www.shopify.com/blog/global-e-commerce-sales>. Accessed: Aug 05, 2024.
- [2] Wayfair, "GitHub - wayfair/WANDS," *GitHub*, 2022. <https://github.com/wayfair/WANDS>. (accessed Aug. 05, 2024).
- [3] K. Divya, B. S. Siddhartha, N. M. Niveditha, B. M. Divya, "(PDF) An Interpretation of Lemmatization and Stemming in Natural Language Processing," *ResearchGate*. https://www.researchgate.net/publication/348306833_An_Interpretation_of_Lemmatization_and_Stemming_in_Natural_Language_Processing.
- [4] R. Pramana, Debora, J. J. Subroto, A. A. S. Gunawan, and Anderies, "Systematic Literature Review of Stemming and Lemmatization Performance for Sentence Similarity," in 2022 IEEE 7th International Conference on Information Technology and Digital Applications (ICITDA), Nov. 2022, doi: 10.1109/ICITDA55840.2022.9971451.
- [5] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Improving Text Embeddings with Large Language Models." Accessed: Aug. 04, 2024. [Online]. Available: <https://arxiv.org/pdf/2401.00368>.
- [6] R. Bhatia, "Cross-encoders vs Bi-encoders : A deep-dive into text encoding methods," *Medium*, Jun. 28, 2024. <https://medium.com/@rbhatia46/cross-encoders-vs-bi-encoders-a-deep-dive-into-text-encoding-methods-d9aa890d6ca4>. (accessed Aug. 04, 2024).
- [7] W. Wang, F. Li, D. Hangbo, B. Yang, and M. Zhou, "MINILM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers." Available: <https://arxiv.org/pdf/2002.10957>.
- [8] J. Pardede, M. G. Husada, and R. Riansyah, "Implementasi dan Perbandingan Metode Okapi BM25 dan PLSA pada Aplikasi Informasi Retrieval," Apr. 2021. Accessed: Aug. 06, 2024. [Online]. Available: (1) (PDF) IMPLEMENTASI DAN PERBANDINGAN METODE OKAPI BM25 DAN PLSA PADA APLIKASI INFORMASI RETRIEVAL (researchgate.net).
- [9] S. Oitihya, "Search Relevance Using BM25 Algorithm," *Otee's Notes on Programming*, Nov. 24, 2021. <https://otee.dev/2021/11/24/search-relevance-using-bm-25.html>. (accessed Aug. 06, 2024).
- [10] P. Miesle, "What is Cosine Similarity?," *DataStax*, Sep. 29, 2023. <https://www.datastax.com/guides/what-is-cosine-similarity>. Accessed: Aug. 07, 2024.
- [11] F. Wu, Y. Liu, R. Gazo, B. Bedrich, and X. Qu, "Some Practice for Improving the Search Results of E-commerce." Accessed: Aug. 06, 2024. [Online]. Available: <https://arxiv.org/pdf/2208.00108>.
- [12] OpenAI, "CLIP," *GitHub*, Dec. 10, 2022. <https://github.com/openai/CLIP>. Accessed: Aug. 07, 2024.
- [13] Y. Wang, L. Wang, Y. Li, D. He, and W. Chen, "A Theoretical Analysis of NDCG Type Ranking Measures," 2013. Accessed: Aug. 05, 2024. [Online]. Available: <https://arxiv.org/pdf/1304.6480>.

LAMPIRAN

Enter text query:

Search by Text

Or upload an image for visual search:

Upload Image

Clear Search



Search Results:

- 1 reclining heated full body massage chair
- 2 ceramic bowl reclining massage chair
- 3 reclining heated full body massage chair
- 4 reclining massage chair with ottoman
- 5 reclining heated massage chair
- 6 barberpub salon massage chair
- 7 massage chair
- 8 beauty spa facial salon tattoo adjustable reclining mass
- 9 saturn genuine leather reclining heated massage chair
- 10 faux leather reclining massage chair

Enter text query:

Search by Text

Or upload an image for visual search:

Upload Image

Clear Search



Search Results:

- 1 bungalow rose coastal 17 " serving platter
- 2 mahershala 32-piece bohemian porcelain dinnerware se
- 3 amarillo polish pottery decorative plate
- 4 kalkaska 9 " melamine salad plate
- 5 amarillo polish pottery 3.15 qt . tureen
- 6 fantasia 9 " melamine salad plate
- 7 brenner sunflower fields 16 piece dinnerware set , servic
- 8 tupelo impression melamine salad plate
- 9 peacock garden 16 piece dinnerware set , service for 4
- 10 symmetry 9 " melamine salad plate